

ABM Automation Project — Architecture & Phase Plan

Author: Sunny Dsouza **Status:** V1 shipped (CLI scorer), planning V2+ **Last updated:** May 21, 2026

TL;DR

The ABM automation system is a **7-stage pipeline**: data sources → ingestion → storage → scoring → QA → activation → reporting. V1 (shipped) covers **stage 4 only**: a CLI scorer using Claude Opus 4.7 + web search that produces structured account reports matching Galyna's existing format. The next 6 phases build out the rest of the pipeline incrementally without rewriting the foundation.

What V1 does today

A single-command CLI that scores any company against one of three frameworks:

```
python scorer.py "OrthoIndy" --segment specialties # 30-pt
python scorer.py "Beacon Health System" --segment hs # 27-pt
python scorer.py "Centene" --segment payer # 30-pt
```

Each run: 1. Loads the segment-specific prompt (Galyna's exact framework, no rewrites) 2. Calls Claude Opus 4.7 with adaptive thinking + server-side web search 3. Streams a structured markdown report to terminal 4. Saves to `outputs/<segment>_<company>_<timestamp>.md`

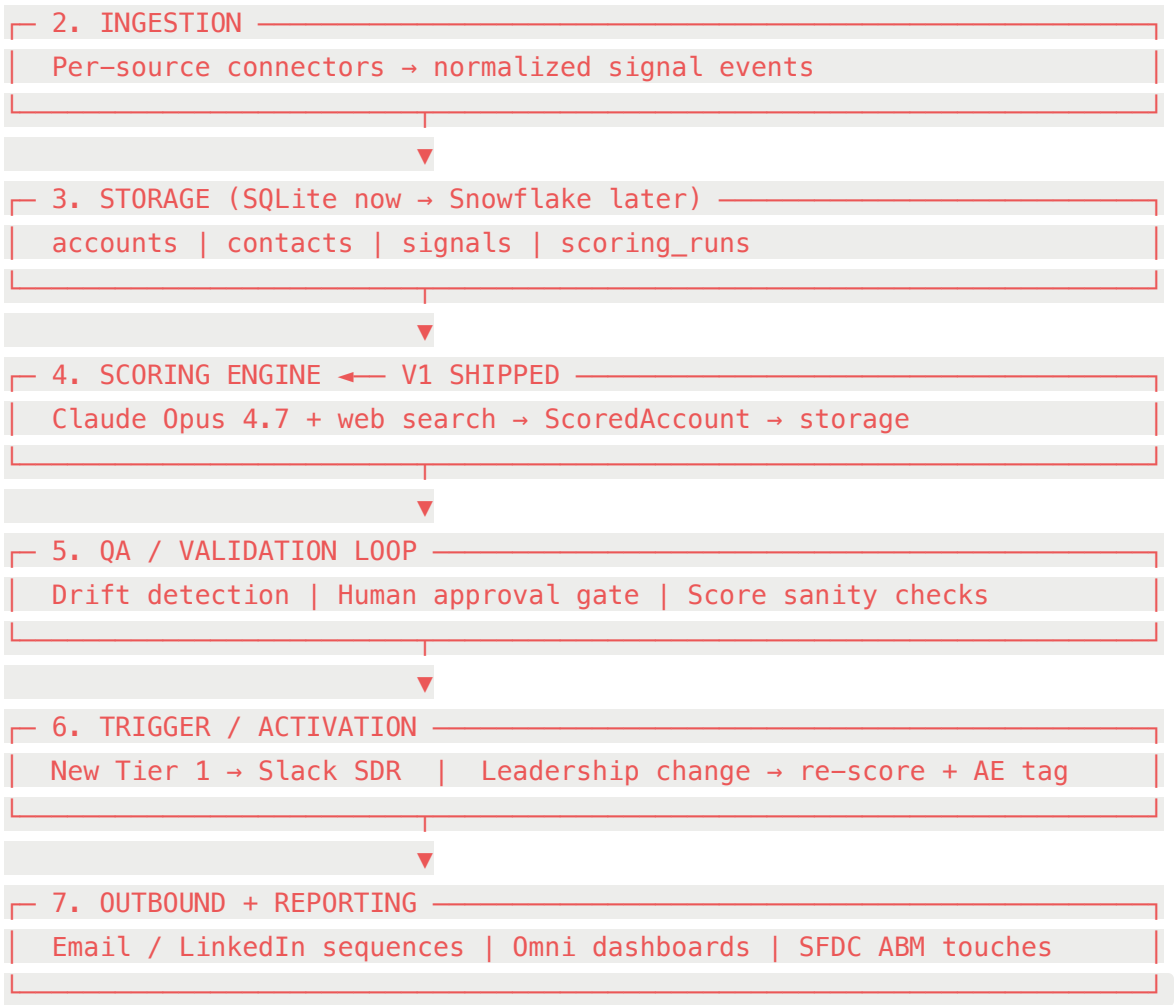
The output has 10 strict sections (Fit Scores → Firmographic Profile → Services → Intent Signals → Decision Makers → Entry Strategy → RCM Complexity → Recent News → Pain Points → Messaging Angles), each rendered as Notion-paste-ready tables and blockquotes.

Verified with: OrthoIndy (25/30 High Fit), Avance Care (24/30 High Fit). Both match Galyna's manual scoring style.

Cost: ~\$0.15–\$0.40 per account. ~\$15–\$40 per 100 accounts.

The full system (where V1 fits, what's still missing)





V1 = box 4 only. Phases 2–6 build out the other six boxes one at a time.

Phase plan

Each phase is independently shippable and adds one capability without breaking what came before.

Phase	What it delivers	New external deps	Effort
1. Foundation refactor	Modular package, Pydantic schemas, SQLite (4 tables), retries, JSON-emit from Claude	None	1 session

Phase	What it delivers	New external deps	Effort
2. First signal pipeline (vertical slice)	One signal source (Apollo OR Apify), end-to-end: ingest → store → re-score → Slack notification	Apollo or Apify, Slack webhook	1–2 sessions
3. Activation layer	Tier-1 alerts, re-scoring triggers, SDR + AE Slack channels	Slack	1 session
4. More signal sources	Add Definitive, Crunchbase, Google News, LinkedIn Ads connectors (one per session)	Each respective API	4–5 sessions
5. Snowflake migration + Omni	Move audit data to Snowflake, hook up Omni dashboards	Snowflake account, Omni	2 sessions
6. Outbound automation	Triggered email / LinkedIn sequences on signal events; SFDC ABM touch logging	Email API (Apollo/Outreach), LinkedIn API, Salesforce	3–4 sessions

QA validation (stage 5 of the diagram) gets folded in across phases — we start simple (re-score + log) and add gates as we see real failure modes.

Phase 1 — Foundation refactor (next session)

The foundation everything else plugs into. No new features, but it transforms V1 from "one script" to "a system."

Directory structure (target end of Phase 1)

```
abm-scorer/
├── pyproject.toml          # replaces requirements.txt
├── README.md
├── .env / .env.example
├── prompts/
└── └── specialties.txt
```

```

|   |— payers.txt
|   |— health_systems.txt
|— abm_scorer/                  # package, flat layout
|   |— __init__.py
|   |— config.py                # env, paths, model settings
|   |— models.py                # Pydantic: storage + content models
|   |— prompts.py               # load_prompt(segment, company)
|   |— client.py                # Anthropic wrapper, retries
|   |— output.py                # normalize_table_cells, save_markdown
|   |— storage.py               # SQLite: init + CRUD for 4 tables
|   |— cli.py                   # argparse entry
|— scorer.py                    # 5-line shim, command unchanged
|— tests/
|   |— test_normalize.py
|   |— test_schema.py
|— outputs/                     # markdown files
|— data/abm.db                  # SQLite, gitignored

```

Data model (4 tables)

```

CREATE TABLE accounts (
  id          INTEGER PRIMARY KEY,
  name        TEXT NOT NULL,
  segment     TEXT NOT NULL CHECK (segment IN ('specialties','payer','hs')),
  domain      TEXT,
  first_seen_at  TIMESTAMP NOT NULL DEFAULT CURRENT_TIMESTAMP,
  last_scored_at  TIMESTAMP,
  current_tier   TEXT,
  current_score  REAL,
  UNIQUE(name, segment)
);

```

```

CREATE TABLE contacts (
  id          INTEGER PRIMARY KEY,
  account_id  INTEGER NOT NULL REFERENCES accounts(id),
  name        TEXT NOT NULL,
  role        TEXT NOT NULL,
  email       TEXT,
  linkedin_url  TEXT,
  is_primary  INTEGER NOT NULL DEFAULT 0,
  source      TEXT NOT NULL,      -- 'scoring_run' | 'apollo' | 'manual'
  observed_at  TIMESTAMP NOT NULL DEFAULT CURRENT_TIMESTAMP

```

```
);
```

```
CREATE TABLE signals (  
  id          INTEGER PRIMARY KEY,  
  account_id  INTEGER NOT NULL REFERENCES accounts(id),  
  signal_type  TEXT NOT NULL,      -- see SignalType enum  
  source       TEXT NOT NULL,      -- see SignalSource enum  
  title        TEXT NOT NULL,  
  payload      TEXT NOT NULL,      -- JSON blob  
  url          TEXT,  
  observed_at  TIMESTAMP NOT NULL,  
  processed_at TIMESTAMP          -- NULL until acted on  
);
```

```
CREATE TABLE scoring_runs (  
  id          INTEGER PRIMARY KEY,  
  account_id  INTEGER NOT NULL REFERENCES accounts(id),  
  total_score  REAL NOT NULL,  
  max_score   INTEGER NOT NULL,  
  tier         TEXT NOT NULL,  
  raw_markdown TEXT NOT NULL,  
  structured_json TEXT NOT NULL,    -- serialized ScoredAccount  
  model       TEXT NOT NULL,  
  cost_usd    REAL,  
  input_tokens INTEGER,  
  output_tokens INTEGER,  
  stop_reason  TEXT,  
  parse_failed INTEGER NOT NULL DEFAULT 0,  
  created_at   TIMESTAMP NOT NULL DEFAULT CURRENT_TIMESTAMP  
);
```

```
CREATE INDEX idx_runs_account    ON scoring_runs(account_id, created_at DESC);  
CREATE INDEX idx_signals_account ON signals(account_id, observed_at DESC);  
CREATE INDEX idx_signals_pending ON signals(processed_at);  
CREATE INDEX idx_contacts_account ON contacts(account_id);
```

All types are Snowflake-portable. Migration in Phase 5 = schema rewrite + `INSERT ... SELECT` from local.

Pydantic schemas (two layers)

Storage models — one per table: - `Account` — persistent record of a company - `Contact` — a person at an account - `Signal` — an observed event tied to an account - `ScoringRun` — one scoring

execution

Content models — nested structure inside `ScoringRun.structured_json` : - `ScoredAccount` (top-level) - `FitScore` , `IntentSignal` , `DecisionMaker` , `ComplexityFactor` , `NewsItem` , `EntryStrategy`

How Claude produces structured JSON

Each prompt gets a final instruction: "After the markdown report, append a `json` code block with this exact schema." We parse the JSON block out and validate against Pydantic `ScoredAccount` . If validation fails, we still save the run with `parse_failed=1` and the raw markdown — never lose data.

Integration map (which phase touches which system)

External system	Phase	Purpose
Anthropic API	1 (done)	Scoring engine
Slack webhook	2	First notification of Tier 1
Apollo / Apify	2	First signal source (leadership change or job posts)
Crunchbase	4	Funding round signals
Google News RSS	4	Press release signals
LinkedIn Ads API	4	Ad engagement signals
Definitive Health	4	Account enrichment, EHR data
Snowflake	5	Production data warehouse
Omni	5	BI dashboards
Salesforce	6	ABM touch logging, contact sync
Email / LinkedIn sequence API	6	Outbound automation

QA validation loop — open design

Placeholder. We don't yet know which failure modes matter most. Probable building blocks:

- **Re-scoring on signal change** — when a Signal lands, re-score the affected account and compare to the last `scoring_runs` row. Log score deltas.
- **Drift detection** — quarterly re-score of all Tier 1 accounts. Alert if score changes by >X points.

- **Human approval gate** — high-impact actions (e.g. triggering outbound) require Galyna's approval before they execute. Easy to add as a `pending_approval` boolean on signal rows.
- **Sample audit** — pick 5% of scoring runs at random for manual review.

We'll harden this once Phase 2–3 are in production and real failure modes appear.

Open questions for Galyna

These are decisions / access requests that block specific phases:

Question	Blocks	Notes
Apollo or Apify access for Phase 2 vertical slice? Which one first?	Phase 2	Apollo = contact + leadership changes; Apify = LinkedIn job posts + profile updates
Which Slack channel should Tier 1 alerts post to? <code>#abm</code> ?	Phase 2/3	Need webhook URL or bot token
Definitive API access — confirmed timeline? Does it include EHR switch data?	Phase 4	If no EHR data, we substitute web search-based detection
LinkedIn Ads API access for paid-click engagement signals?	Phase 4	Confirmed in the project doc as  — need credentials
Claude company account API access (vs my personal key)	Operational	Today we run on Sunny's key with rotation needed
Snowflake vs. continued SQLite — what's the timeline?	Phase 5	SQLite works fine for <10K accounts. Migration is straightforward.
Scoring drift tolerance — what counts as "Galyna needs to review"?	QA design	Probably ± 3 points or tier change, but should validate with real data
Are existing ABM sheet (Google Sheets) records the seed data, or do we start fresh from Definitive?	Phase 1/2	Affects initial <code>accounts</code> table population

What you can demo on Friday

1. **The CLI in action** — score one new account live (e.g. Indiana Physical Therapy or any account Galyna names) — ~2 minutes
2. **Pre-generated examples** — OrthoIndy, Avance Care reports as Notion-paste examples

3. **This architecture doc** — walk through the diagram + phase plan; ask Galyna to react

4. **Open questions list** — get her decisions on the table

What you're *not* demoing yet: signal ingestion, Slack alerts, outbound, dashboards. Those are Phase 2+.

Next session

Phase 1 implementation. Estimated ~440 lines across 7 focused modules, all behavior-preserving (the existing `python scorer.py ...` command works identically). Steps:

1. `pyproject.toml` + package skeleton (empty modules)
2. Move existing code → modules, no behavior change
3. Add `models.py` with both layers of Pydantic models
4. Add `storage.py` + initialize SQLite
5. Update prompts to append JSON block
6. Wire `client.py` to parse JSON + validate against schema
7. Wire `cli.py` to call `storage.save_run()` after each run
8. Minimal tests (`test_normalize` , `test_schema`)

Each step commits independently. CLI keeps working at every step.